Python-Activity-Based-Learning

Test Case-1

1.University Scholarship Allocation System using Python

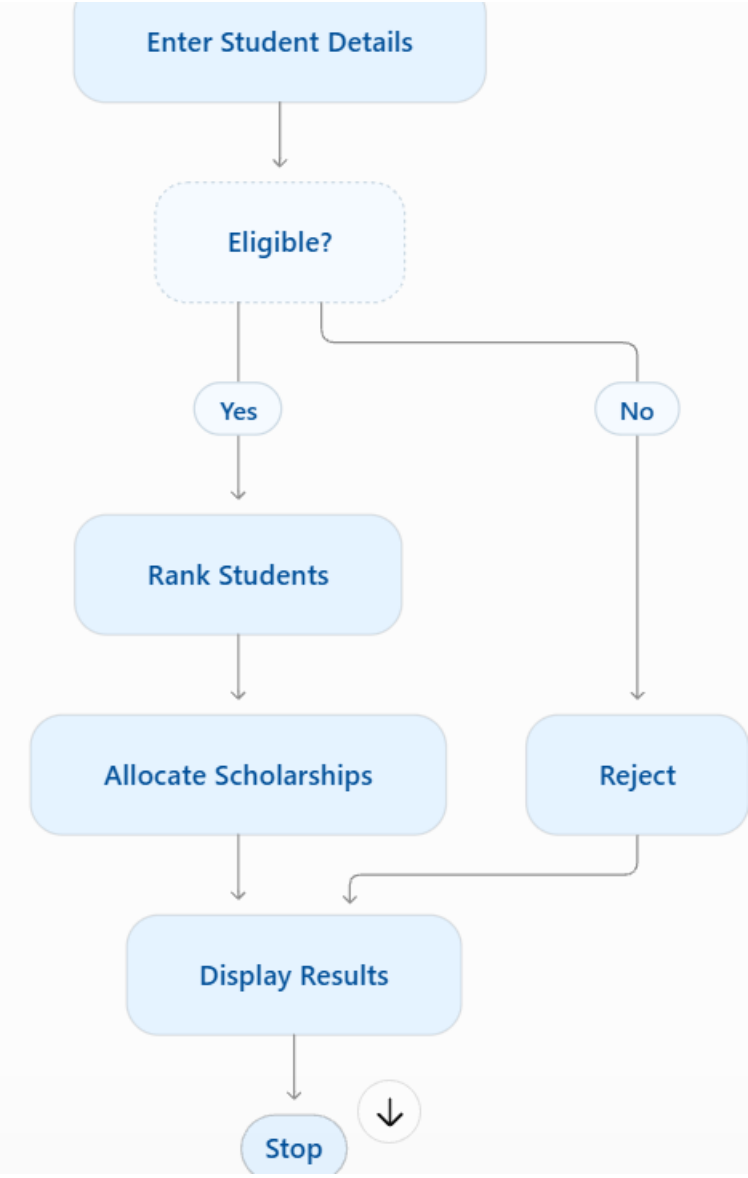
1. Problem Statement

The University Scholarship Allocation System evaluates students based on aggregate marks, subject-wise minimum marks, attendance percentage, and family annual income. It identifies eligible students and ranks them according to aggregate marks. If two students have equal marks, attendance is used as the first tie-breaker, followed by lower family income. The system displays the final ranked list of eligible students.

2. Algorithm

1. Start the program.
2. Read the number of applicants.
3. Enter each student's name, aggregate marks, subject marks, attendance percentage, and annual family income.
4. Check the eligibility criteria:
    - Aggregate marks must be at least 75%.
    - Every subject mark must be at least 60%.
    - Attendance must be at least 75%.
    - Annual family income must not exceed ₹2,50,000.
5. Add eligible students to the eligible list.
6. Sort eligible students by:
    - Higher aggregate marks first.
    - Higher attendance first if aggregate marks are equal.
    - Lower family income first if both are equal.
7. Display the ranked list of eligible students.
8. If no students qualify, display a suitable message.
9. Stop the program.

3. Flowchart



5. Sample Input

Enter number of applicants: 3

Applicant 1 Enter student name: Ravi Enter aggregate marks (%): 85 Enter marks in 3 subjects: 80 90 75 Enter attendance (%): 90 Enter annual family income: Rs. 200000

Applicant 2 Enter student name: Sita Enter aggregate marks (%): 85 Enter marks in 3 subjects: 75 80 70 Enter attendance (%): 95 Enter annual family income: Rs. 220000

Applicant 3 Enter student name: Aman Enter aggregate marks (%): 70 Enter marks in 3 subjects: 65 70 75 Enter attendance (%): 80 Enter annual family income: Rs. 150000

6. Sample Output

Applicant 1 Eligible for scholarship

Applicant 2 Eligible for scholarship

Applicant 3 Not eligible for scholarship

--- Scholarship Merit List ---

1. Sita - Marks: 85.0%, Attendance: 95.0%, Income: Rs.220000.0
2. Ravi - Marks: 85.0%, Attendance: 90.0%, Income: Rs.200000.0

Test Case-2

2. Progressive Electricity Billing System

1. Problem Statement

Write a Python program to calculate monthly household electricity bills using different consumption slabs, fixed charges, surcharges, and rebates. Compare bills across multiple months and identify unusual increases.

Billing Criteria (Example):

Units Consumed	Rate per Unit
First 100 units	₹2
Next 100 units	₹3
Above 200 units	₹5

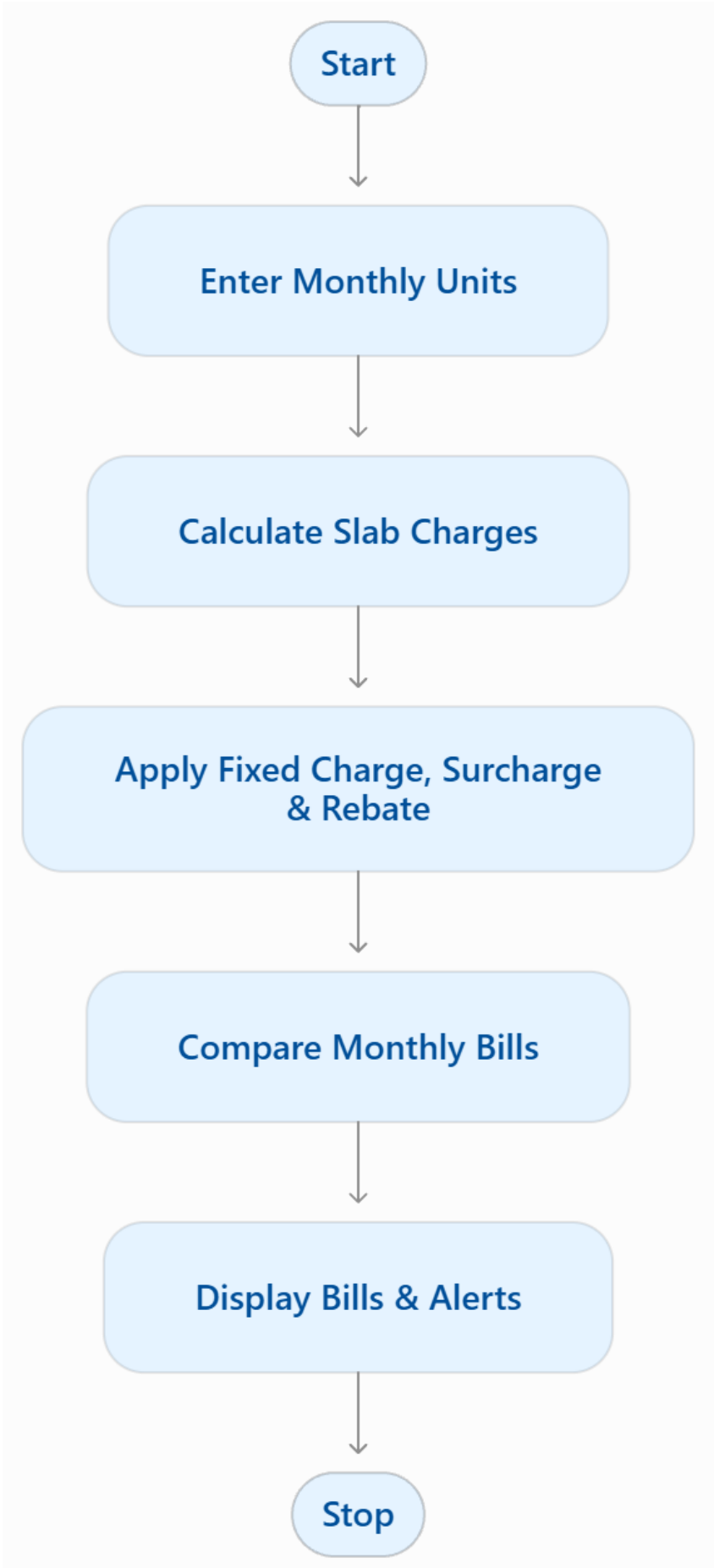
- Fixed charge: ₹50 per month
- Surcharge: 5% if consumption exceeds 300 units
- Rebate: 10% if consumption is 100 units or less
- Unusual increase: Bill increases by more than 30% compared to the previous month.

2. Algorithm

1. Start the program.
2. Enter the number of months and units consumed.
3. Calculate charges using electricity slabs.
4. Add fixed charges and apply surcharge or rebate.
5. Calculate the final monthly bill.
6. Compare it with the previous month's bill.
7. Display bills and identify unusual increases.

8. Stop.

3. Flowchart


Start
Enter Monthly Units
Calculate Slab Charges
Apply Fixed Charge, Surcharge & Rebate
Compare Monthly Bills
Display Bills & Alerts
Stop

5. Sample Input

Enter number of months: 3

Month 1 Enter units consumed: 80

Month 2 Enter units consumed: 150

Month 3 Enter units consumed: 350

6. Sample Output

Month 1 Bill: Rs. 189.0

Month 2 Bill: Rs. 400.0 Alert: Unusual bill increase!

Month 3 Bill: Rs. 840.0 Alert: Unusual bill increase!

Monthly Bill Summary Month 1 : Rs. 189.0 Month 2 : Rs. 400.0 Month 3 : Rs. 840.0

Bill Calculation

Month	Units	Final Bill
1	80	₹189
2	150	₹400
3	350	₹840

Explanation: Month 2 and Month 3 show unusual increases because their bills exceed the previous

Test Case-3

3. Hospital Billing and Patient Classification

1. Problem Statement

Write a Python program to calculate hospital bills using consultation, diagnostic, and treatment charges. Apply concessions based on patient category, validate billing amounts, and classify patients according to their final bill.

Assumed Payment Policy:

Patient Category	Concession
General	0%
Senior Citizen	20%
Low Income	30%

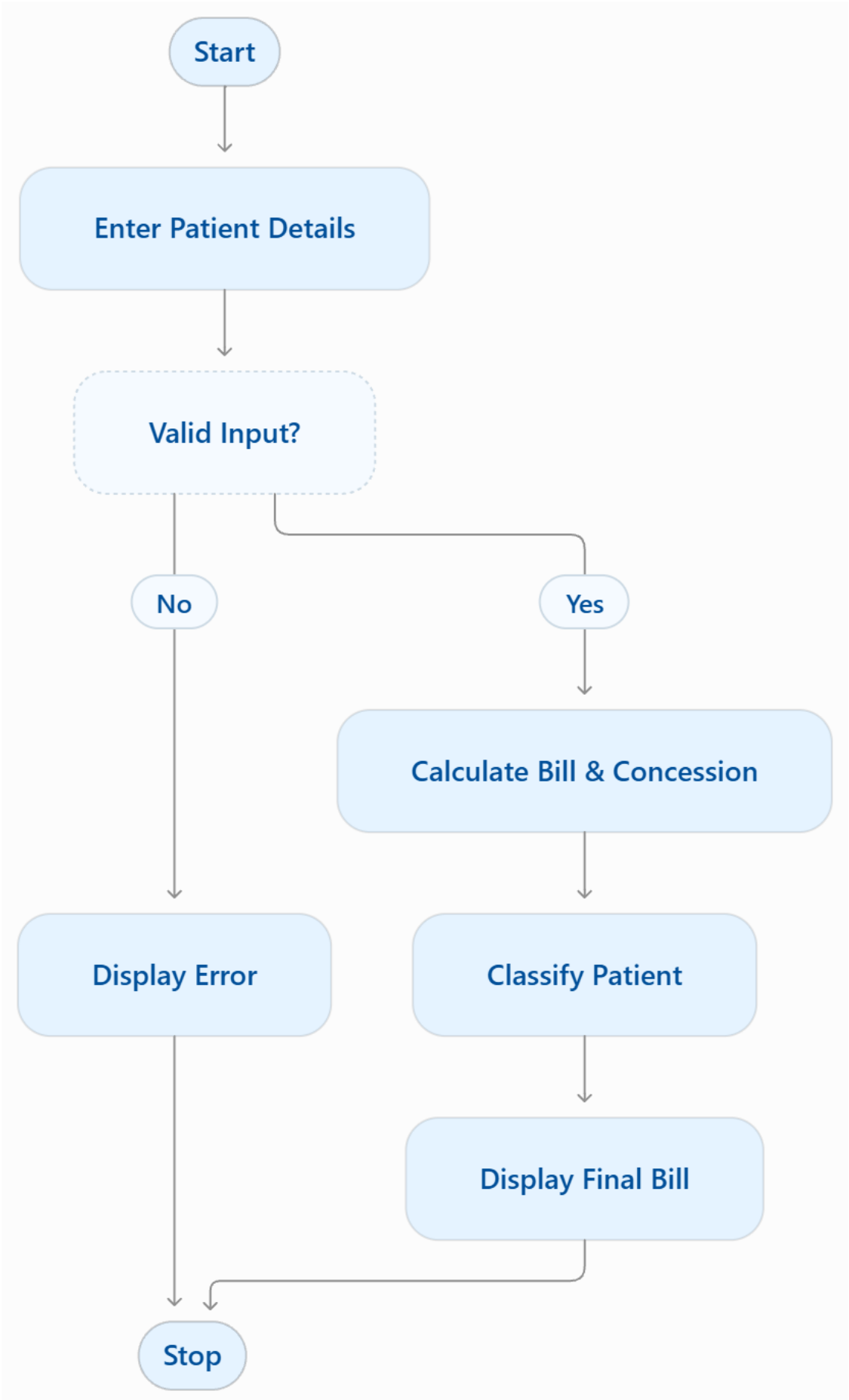
Patient Classification:

Final Bill	Classification
Below ₹2,000	Low Bill
₹2,000–₹10,000	Medium Bill
Above ₹10,000	High Bill

2. Algorithm

1. Start the program.
2. Enter patient name, category, and hospital charges.
3. Validate that all charges are non-negative.
4. Calculate the total hospital bill.
5. Apply concession based on patient category.
6. Calculate the final payable amount.
7. Classify the patient based on the final bill.
8. Display the bill and stop.

3.Flowchart


Start
Enter Patient Details
Valid Input?
No
Yes
Calculate Bill & Concession
Display Error
Classify Patient
Display Final Bill
Stop

5. Sample Input

Enter patient name: Ravi

1.General 2.Senior Citizen 3.Low Income Enter category (1-3): 2

Consultation charge: 1000 Diagnostic charge: 2000 Treatment charge: 5000

6. Sample Output

--- Hospital Bill ---

Patient: Ravi Category: Senior Citizen Total: Rs. 8000.0 Concession: Rs. 1600.0 Final Bill: Rs. 6400.0 Classification: Medium Bill

Calculation

Total Bill = 1000 + 2000 + 5000 = ₹8,000

Concession = 8000 × 20% = ₹1,600

Final Bill = 8000 − 1600 = ₹6,400

Therefore, the patient is classified under the Medium Bill category.

Test Case-4

4. Employee Payroll and Deduction Analysis

1. Problem Statement

Write a Python program to calculate an employee's gross salary using basic pay, allowances, overtime, and incentives. Apply progressive deductions and generate a formatted salary statement showing gross salary, total deductions, and net salary.

Assumed Deduction Policy:

Gross Salary Slab	Deduction Rate
First ₹20,000	5%
Next ₹30,000	10%
Above ₹50,000	15%

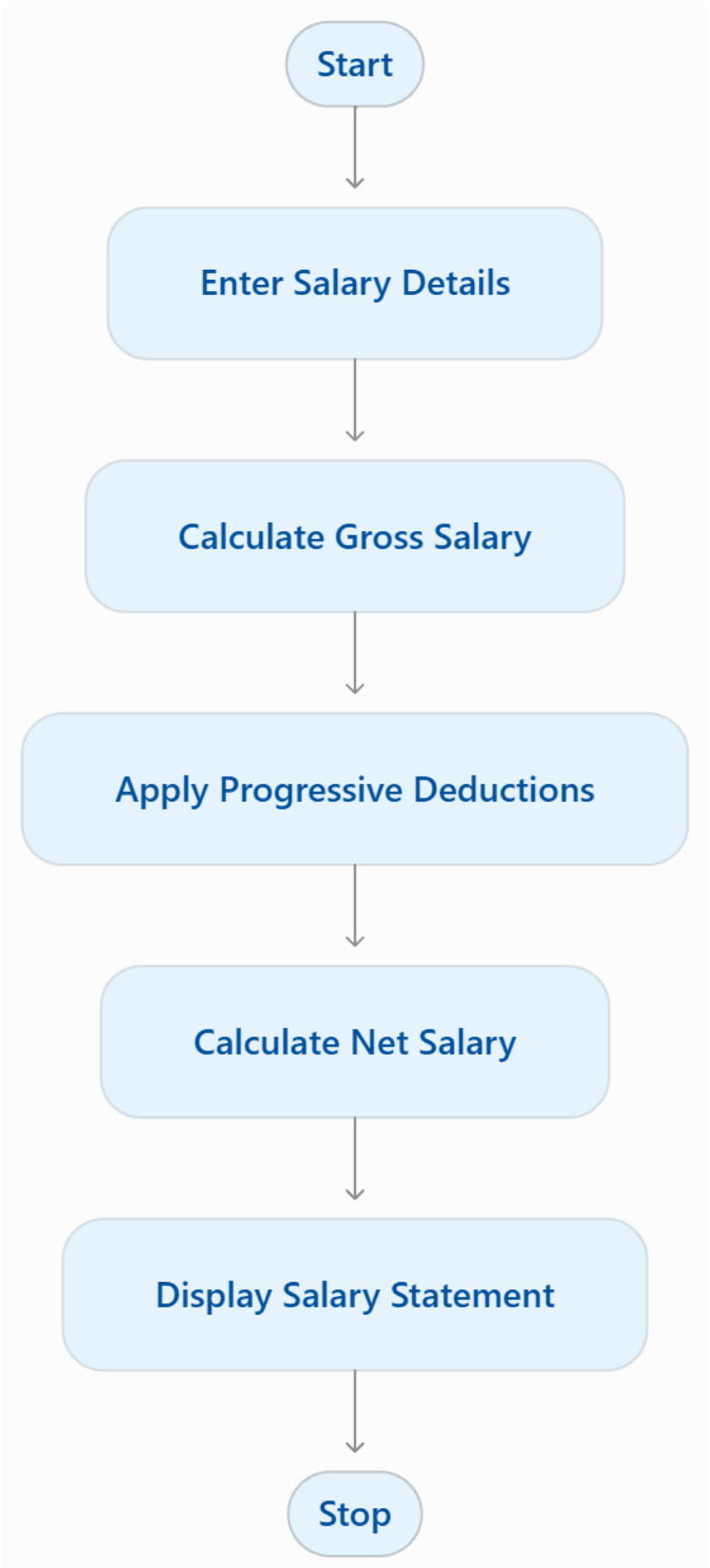
Gross Salary = Basic Pay + Allowances + Overtime Pay + Incentives

Net Salary = Gross Salary − Total Deductions

2. Algorithm

1. Start the program.
2. Enter employee name and salary details.
3. Calculate overtime pay.
4. Calculate gross salary by adding all earnings.
5. Apply progressive deductions based on salary slabs.
6. Calculate net salary.
7. Display the formatted salary statement.
8. Stop.

3. Flowchart


Start
Enter Salary Details
Calculate Gross Salary
Apply Progressive Deductions
Calculate Net Salary
Display Salary Statement
Stop

5. Sample Input

Employee Name: Ravi Basic Pay: 40000 Allowances: 8000 Overtime Hours: 10 Overtime Rate: 200 Incentives: 5000

6. Sample Output

------ SALARY STATEMENT ------ Employee Name : Ravi Basic Pay : Rs.40000.00 Allowances : Rs.8000.00 Overtime Pay : Rs.2000.00 Incentives : Rs.5000.00

Gross Salary : Rs.55000.00 Deductions : Rs.4750.00 Net Salary : Rs.50250.00

Calculation

Gross Salary:

40,000 + 8,000 + 2,000 + 5,000 = ₹55,000

Progressive Deductions:

Salary Portion	Deduction
First ₹20,000 × 5%	₹1,000
Next ₹30,000 × 10%	₹3,000
Remaining ₹5,000 × 15%	₹750
Total Deduction	₹4,750

Net Salary = ₹55,000 − ₹4,750 = ₹50,250

TestCase-5

5. Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Write a Python program to compare multiple mobile plans based on data usage, validity, and budget. Calculate extra usage charges and identify the most economical eligible plan.

Assumed Mobile Plans:

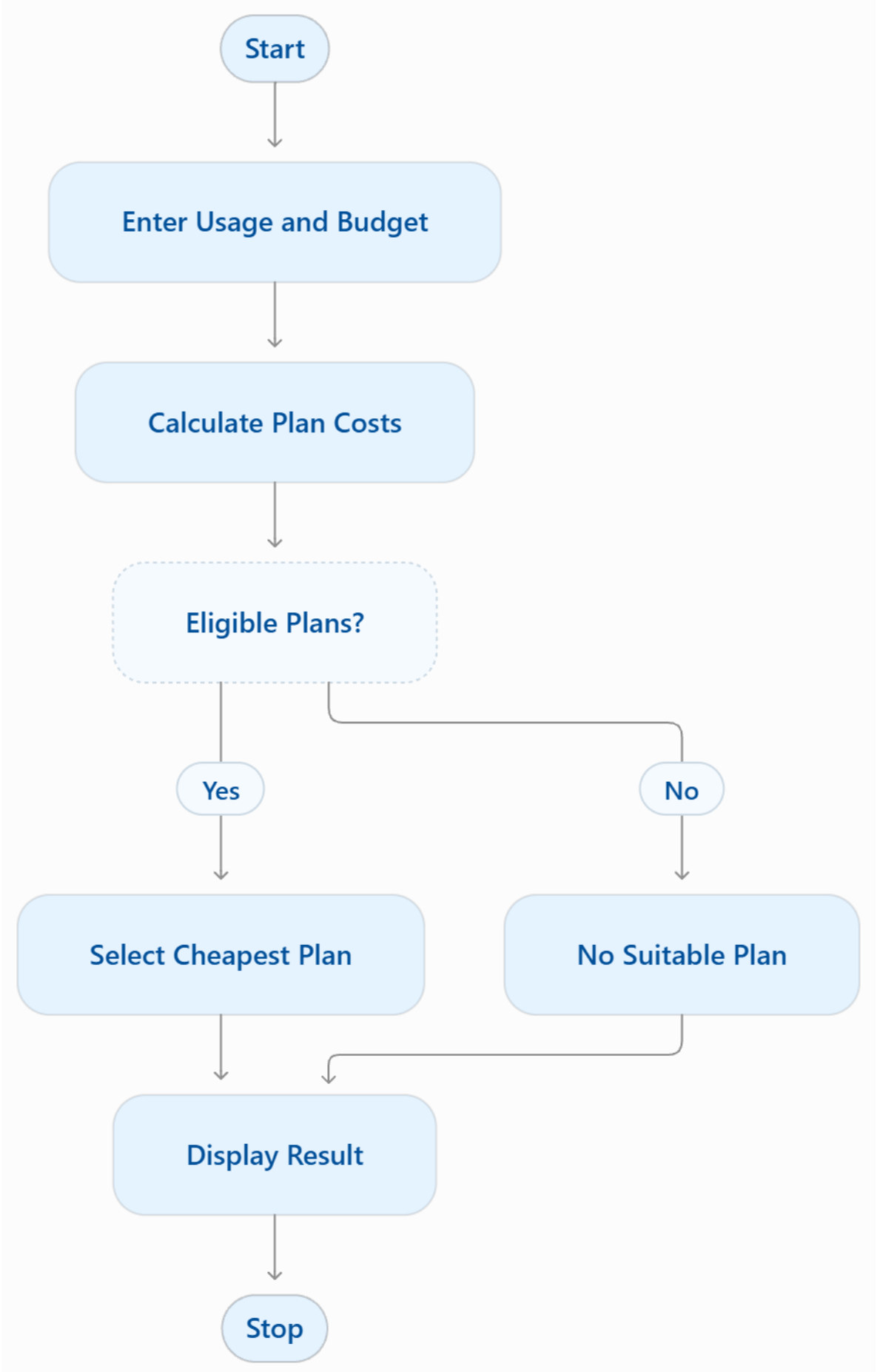
Plan	Price	Data Limit	Validity	Extra Charge/GB
Basic	₹199	2 GB	28 days	₹50
Standard	₹399	5 GB	30 days	₹40
Premium	₹599	10 GB	56 days	₹30

The best plan must satisfy the required validity and budget while having the lowest total cost.

2. Algorithm

1. Start the program.
2. Enter required data, validity, and budget.
3. Store the available mobile plans.
4. Calculate excess data usage for each plan.
5. Add excess charges to the base price.
6. Check validity and budget constraints.
7. Select the eligible plan with the lowest total cost.
8. Display the best plan and stop.

3. Flowchart


Start
Enter Usage and Budget
Calculate Plan Costs
Eligible Plans?
Yes
No
Select Cheapest Plan
No Suitable Plan
Display Result
Stop

5. Sample Input

Required Data (GB): 6 Required Validity (days): 30 Enter Budget: 700

6. Sample Output

--- Plan Comparison ---

Plan: Basic Extra Charge: Rs. 200.0 Total Cost: Rs. 399.0 Status: Not Eligible

Plan: Standard Extra Charge: Rs. 40.0 Total Cost: Rs. 439.0 Status: Eligible

Plan: Premium Extra Charge: Rs. 0.0 Total Cost: Rs. 599.0 Status: Eligible

--- Best Plan --- Recommended Plan: Standard Minimum Cost: Rs. 439.0

Calculation

Plan	Total Cost	Status
Basic	₹399	Not Eligible (Validity)
Standard	₹439	Eligible
Premium	₹599	Eligible

Standard Plan is selected because it satisfies the required data, validity, and budget constraints at the

Test Case-6

19. Multi-Product Shopping Budget Constraint

1. Problem Statement

Write a Python program to calculate the total cost of multiple products based on their prices and quantities. Apply discounts and check whether the shopping bill is within the given budget. If the bill exceeds the budget, suggest reducing product quantities.

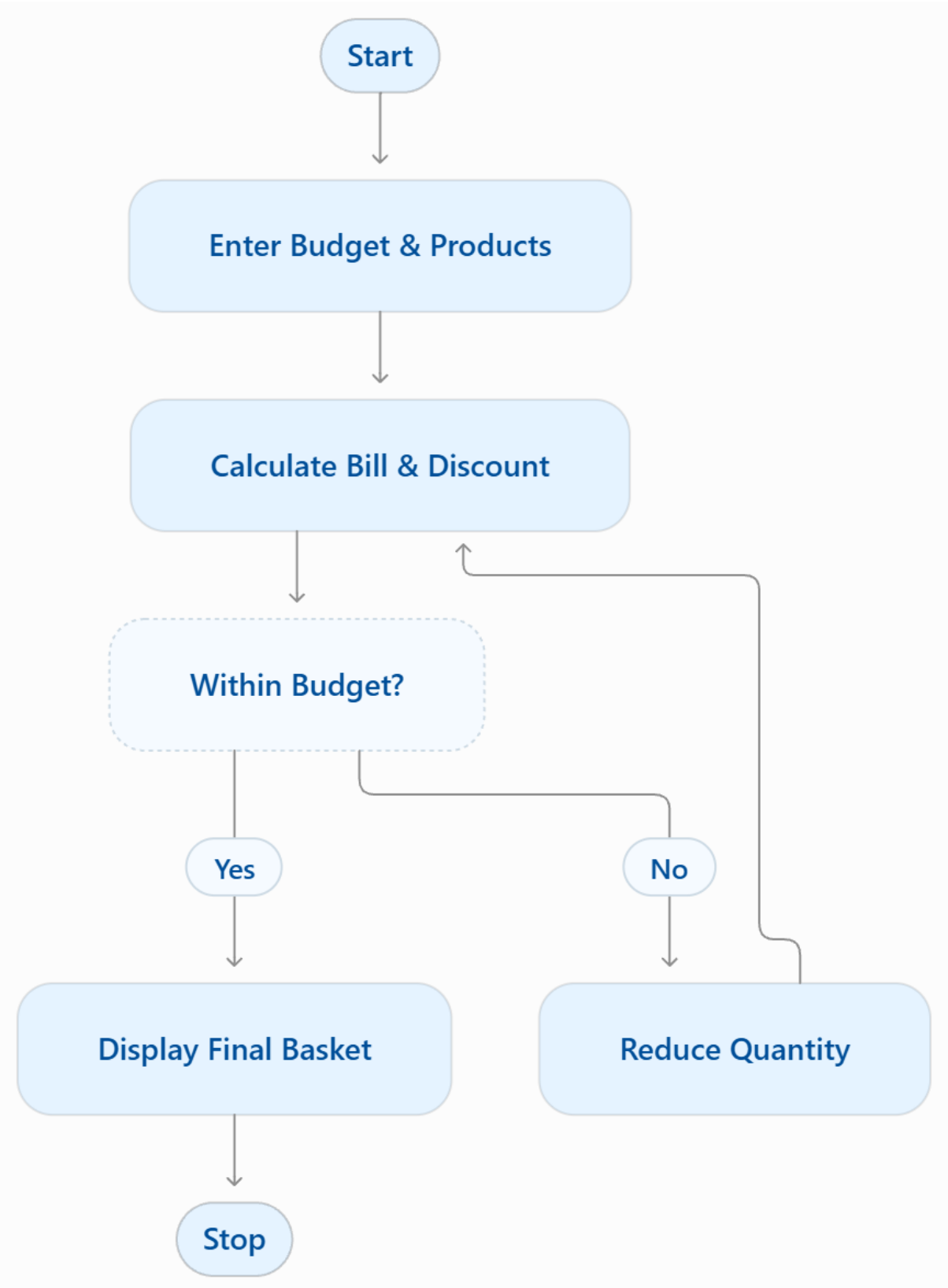
Assumed Discount Policy:

- Total of ₹2,000 or above: 10% discount.
- Total below ₹2,000: No discount.
- If the final bill exceeds the budget, reduce the quantity of the most expensive available product first.

2. Algorithm

1. Start the program.
2. Enter the budget and product details.
3. Calculate the total shopping cost.
4. Apply discount if applicable.
5. Compare the final bill with the budget.
6. If the bill exceeds the budget, reduce product quantities.
7. Recalculate the bill until it fits the budget.
8. Display the final basket and stop.

3. Flowchart



5. Sample Input

Enter budget: 1800 Number of products: 3

Product name: Rice Price: 600 Quantity: 2

Product name: Oil Price: 500 Quantity: 2

Product name: Sugar Price: 100 Quantity: 2

6. Sample Output

Original Bill: Rs. 2160.0

Reduced 1 quantity of Rice

--- Final Shopping Basket --- Rice - Quantity: 1 Oil - Quantity: 2 Sugar - Quantity: 2

Total: Rs. 1800.0 Discount: Rs. 0 Final Bill: Rs. 1800.0 Remaining Budget: Rs. 0.0

Calculation

Product	Original Quantity	Final Quantity
Rice	2	1
Oil	2	2
Sugar	2	2

Original Total = ₹2,400

Discount (10%) = ₹240

Original Bill = ₹2,160

After reducing one quantity of Rice:

Final Bill = ₹1,800

Therefore, the shopping basket is within the given budget.

Test Case-7

51. Bus Passenger Occupancy Simulation

1. Problem Statement

Write a Python program to simulate bus passenger occupancy across multiple stops. The program accepts boarding and alighting counts, calculates the number of passengers after each stop, and identifies capacity violations.

Conditions:

- Bus capacity is entered by the user.

- Passenger count cannot be negative.
- Alighting passengers cannot exceed the current occupancy.
- If occupancy exceeds capacity, display a warning.

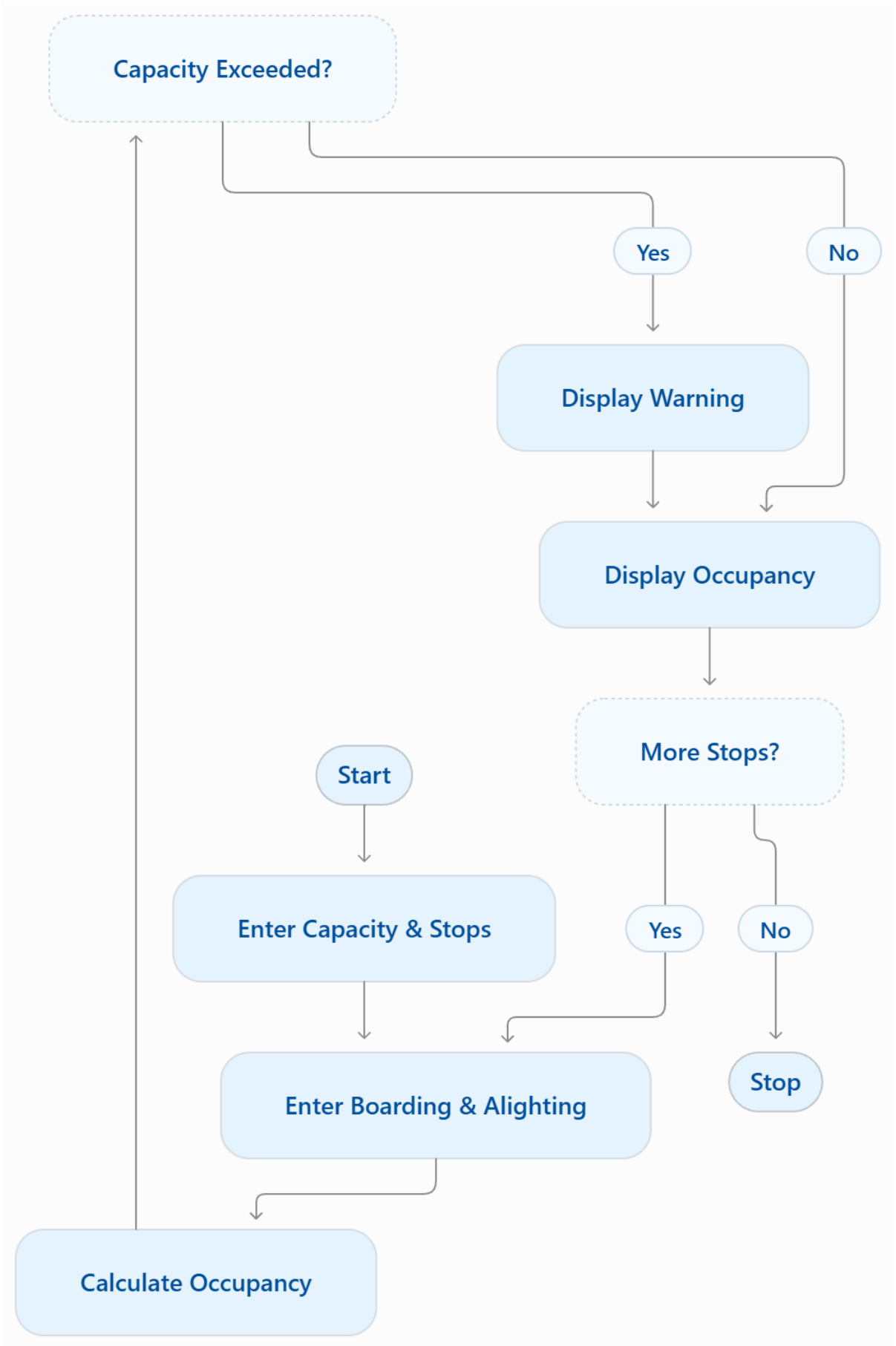
2. Algorithm

1. Start the program.
2. Enter bus capacity and number of stops.
3. Initialize passenger count to zero.
4. Enter boarding and alighting counts at each stop.
5. Validate the passenger counts.
6. Update occupancy using:

Occupancy = Previous Occupancy − Alighting + Boarding

7. Check whether occupancy exceeds capacity.
8. Display occupancy and warnings.
9. Stop.

3. Flowchart


Capacity Exceeded?
Yes
No
Display Warning
Display Occupancy
More Stops?
Start
Enter Capacity & Stops
Yes
No
Stop
Enter Boarding & Alighting
Calculate Occupancy

5. Sample Input

Enter bus capacity: 40 Enter number of stops: 4

Stop 1 Passengers boarding: 20 Passengers alighting: 0

Stop 2 Passengers boarding: 15 Passengers alighting: 5

Stop 3 Passengers boarding: 20 Passengers alighting: 5

Stop 4 Passengers boarding: 5 Passengers alighting: 15

6. Sample Output

Stop 1 Current Occupancy: 20 Status: Within capacity

Stop 2 Current Occupancy: 30 Status: Within capacity

Stop 3 Current Occupancy: 45 Warning: Capacity exceeded!

Stop 4 Current Occupancy: 35 Status: Within capacity

----- BUS SUMMARY ----- Final Occupancy: 35 Capacity Violations: 1 Simulation Completed

Occupancy Calculation

Stop	Boarding	Alighting	Occupancy	Status
1	20	0	20	Normal
2	15	5	30	Normal
3	20	5	45	Capacity Exceeded
4	5	15	35	Normal

At Stop 3, the passenger count reaches 45, exceeding the bus capacity of 40 by 5 passengers.

Test Case-8

83. Food Supply Chain Stock Reconciliation

1. Problem Statement

Write a Python program to manage food stock across multiple locations. The system compares received, distributed, damaged, and remaining quantities, identifies stock discrepancies, and calculates replenishment requirements.

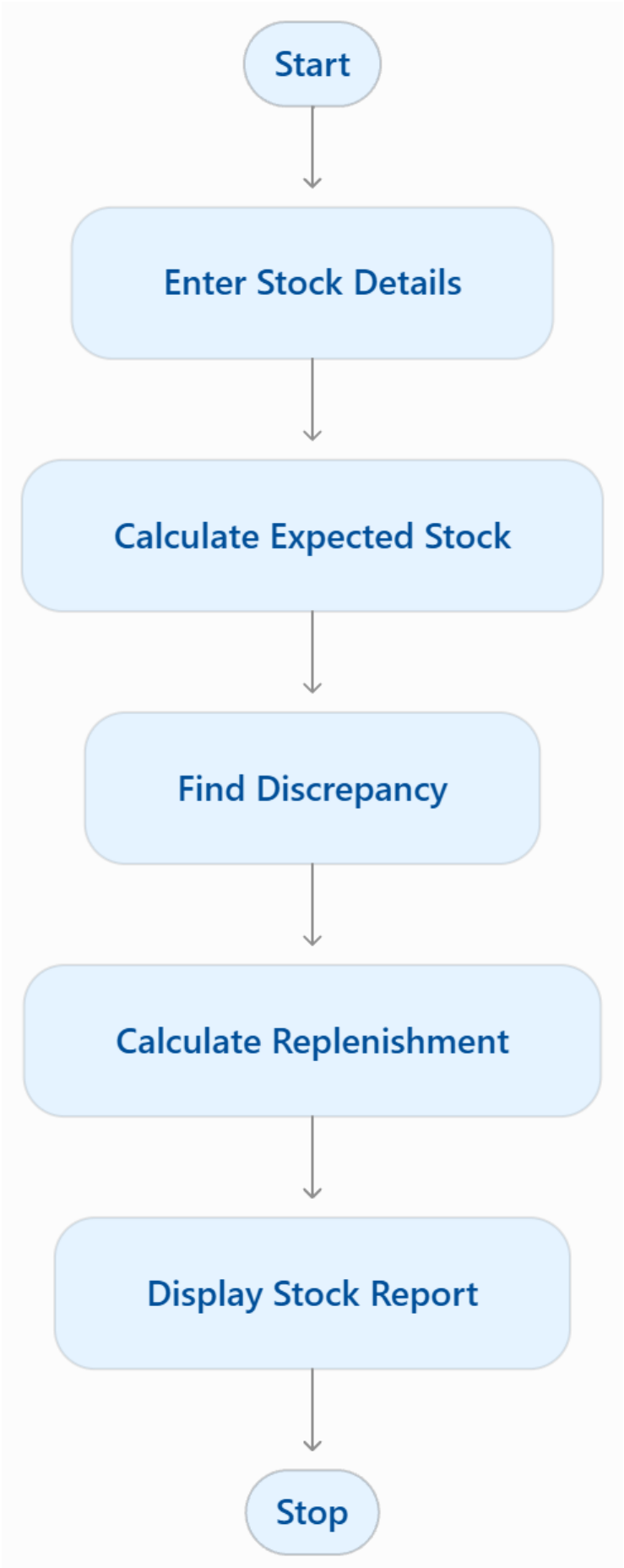
Conditions:

- Expected Stock = Received − Distributed − Damaged
- Discrepancy = Actual Stock − Expected Stock
- Replenishment = Maximum(0, Minimum Required Stock − Actual Stock)
- Negative quantities are not allowed.
- Minimum required stock is entered for each location.

2. Algorithm

1. Start the program.
2. Enter the number of locations.
3. Read received, distributed, damaged, actual stock, and minimum stock.
4. Validate the entered quantities.
5. Calculate expected remaining stock.
6. Compare expected stock with actual stock.
7. Identify stock discrepancies.
8. Calculate replenishment requirements.
9. Display the stock report for each location.
10. Stop.

3. Flowchart


Start
Enter Stock Details
Calculate Expected Stock
Find Discrepancy
Calculate Replenishment
Display Stock Report
Stop

5. Sample Input

Enter number of locations: 2

Location 1 Location name: Warehouse A Stock received: 500 Stock distributed: 300 Stock damaged: 20 Actual remaining stock: 170 Minimum required stock: 200

Location 2 Location name: Warehouse B Stock received: 400 Stock distributed: 250 Stock damaged: 10 Actual remaining stock: 140 Minimum required stock: 180

6. Sample Output

--- Stock Report --- Location: Warehouse A Expected Stock: 180 Actual Stock: 170 Discrepancy: -10 Status: Stock Shortage Replenishment Required: 30

--- Stock Report --- Location: Warehouse B Expected Stock: 140 Actual Stock: 140 Discrepancy: 0 Status: Stock Matched Replenishment Required: 40

--- Final Summary --- Total Replenishment: 70 Total Absolute Discrepancy: 10 Stock Reconciliation Completed

Stock Calculation

Details	**Warehouse A**	**Warehouse B**
Received	500	400
Distributed	300	250
Damaged	20	10
Expected Stock	180	140
Actual Stock	170	140
Discrepancy	-10	0
Replenishment	30	40

Explanation: Warehouse A has a shortage of 10 units, while Warehouse B has no discrepancy. Both warehouses require replenishment to maintain their minimum stock levels.

Test Case-9

97. Smart Campus Incident and Maintenance Prioritization

1. Problem Statement

Write a Python program to classify and prioritize campus maintenance reports based on severity, location, and reported frequency. The system ranks incidents using predefined rules and generates a priority summary.

Priority Rules:

Criteria	Points
High Severity	3
Medium Severity	2
Low Severity	1
Critical Location (Lab/Hostel)	+2
Other Location	+0
Frequency ≥ 3	+2
Frequency below 3	+0

Classification:

- Score ≥ 6: High Priority
- Score 4–5: Medium Priority
- Score below 4: Low Priority

If scores are equal, higher severity, followed by higher reported frequency, gets priority. Remaining ties are resolved alphabetically by location and incident name.

2. Algorithm

1. Start the program.
2. Enter the number of maintenance reports.
3. Read incident name, severity, location, and frequency.

4. Calculate priority score using predefined rules.
5. Classify each incident as High, Medium, or Low.
6. Rank incidents based on score, severity, and frequency.
7. Display the priority summary.
8. Stop.

3. Flowchart


Start
Enter Incident Details
Calculate Priority Score
Classify & Rank Incidents
Display Priority Summary
Stop

5. Sample Input

Enter number of reports: 3

Report 1 Incident name: Water Leakage Severity (High/Medium/Low): High Location: Hostel Reported frequency: 4

Report 2 Incident name: Broken Light Severity (High/Medium/Low): Low Location: Classroom Reported frequency: 2

Report 3 Incident name: Computer Failure Severity (High/Medium/Low): Medium Location: Lab Reported frequency: 3

6. Sample Output

----- PRIORITY SUMMARY -----

1 Water Leakage | Score: 7 | Priority: High 2 Computer Failure | Score: 6 | Priority: High 3 Broken Light | Score: 1 | Priority: Low

Total Reports: 3 High Priority: 2 Medium Priority: 0 Low Priority: 1

Priority Calculation

Incident	Severity	Location	Frequency	Score	Priority
Water Leakage	High	Hostel	4	7	High
Computer Failure	Medium	Lab	3	6	High
Broken Light	Low	Classroom	2	1	Low

Explanation: Water Leakage receives the highest priority because of its high severity, critical location, and repeated

Test Case-10

14. Warehouse Inventory Reconciliation

1. Problem Statement

Write a Python program to manage warehouse inventory across multiple products. The system compares opening stock, incoming stock, dispatched quantities, and actual closing stock. It identifies stock discrepancies and calculates replenishment requirements.

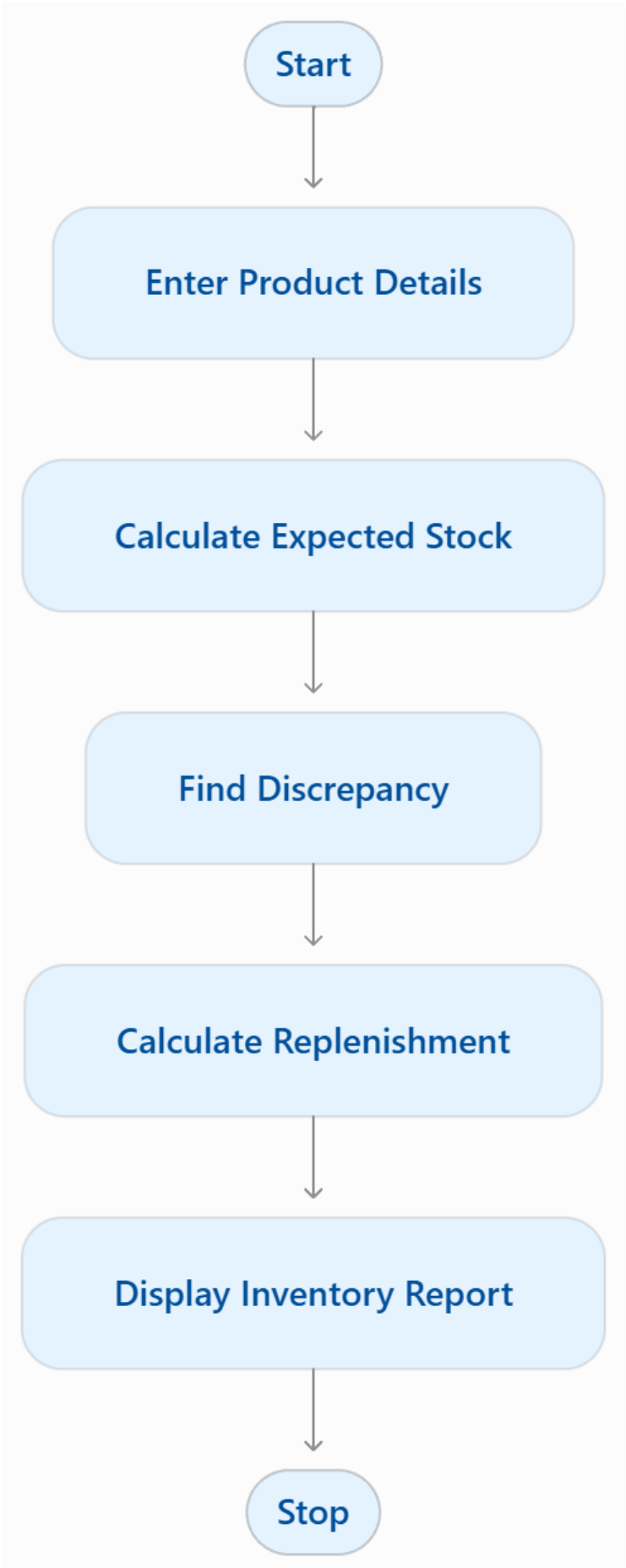
Formulas:

- Expected Stock = Opening Stock + Incoming Stock − Dispatched Stock
- Discrepancy = Actual Closing Stock − Expected Stock
- Replenishment = max(0, Minimum Stock − Actual Closing Stock)

2. Algorithm

1. Start the program.
2. Enter the number of products.
3. Read opening, incoming, dispatched, closing, and minimum stock.
4. Validate the entered quantities.
5. Calculate expected closing stock.
6. Compare expected and actual closing stock.
7. Identify stock shortages or excess stock.
8. Calculate replenishment requirements.
9. Display the inventory summary and stop.

3. Flowchart


Start
Enter Product Details
Calculate Expected Stock
Find Discrepancy
Calculate Replenishment
Display Inventory Report
Stop

5. Sample Input

Enter number of products: 2

Product 1 Product name: Rice Opening stock: 200 Incoming stock: 100 Dispatched stock: 180 Actual closing stock: 110 Minimum required stock: 150

Product 2 Product name: Sugar Opening stock: 150 Incoming stock: 50 Dispatched stock: 100 Actual closing stock: 100 Minimum required stock: 120

6. Sample Output

--- Inventory Report --- Product: Rice Expected Stock: 120 Actual Stock: 110 Discrepancy: -10 Status: Stock Shortage Replenishment Required: 40

--- Inventory Report --- Product: Sugar Expected Stock: 100 Actual Stock: 100 Discrepancy: 0 Status: Stock Matched Replenishment Required: 20

--- Final Summary --- Total Replenishment: 60 Total Absolute Discrepancy: 10 Inventory Reconciliation Completed

Inventory Calculation

Details	**Rice**	**Sugar**
Opening Stock	200	150
Incoming Stock	100	50
Dispatched Stock	180	100
Expected Stock	120	100
Actual Stock	110	100
Discrepancy	-10	0
Replenishment	40	20

Explanation: Rice has a shortage of 10 units, while Sugar has no discrepancy. Both products require replenishment to maintain their minimum stock levels.